Planificación de horarios universitarios mediante una revisión crítica y la reproducción de LectureKick

[Nombre y apellidos de las tres personas]
Máster Universitario en Aprendizaje Automático y Datos Masivos
Universidad Politécnica de Madrid

Resumen

I. Introducción

II. Objetivos y preguntas de investigación

III. Metodología de la revisión bibliográfica

IV. Formulación del problema y benchmarks

V. Estado del arte

El problema de planificación universitaria basada en planes de estudio (Curriculum-Based Course Timetabling, CB-CTT) consiste en asignar cada clase a un periodo y un aula. El horario debe respetar las restricciones obligatorias y reducir las penalizaciones asociadas a las preferencias. Incluso después de obtener una solución factible, el número de alternativas posibles sigue siendo elevado, por lo que buena parte de la literatura recurre a metaheurísticas [1].

V-A. Metaheurísticas

Los primeros trabajos sobre CB-CTT combinaron movimientos de reasignación e intercambio con Hill Climbing y búsqueda tabú. En la Segunda Competición Internacional de Horarios de 2007 (Second International Timetabling Competition, ITC-2007), el método de Müller obtuvo el mejor resultado en la modalidad dedicada a este problema. Su propuesta coordina cinco tipos de movimiento con Hill Climbing, Gran Diluvio (Great Deluge) y enfriamiento simulado [2]. Hill Climbing conserva los cambios que mejoran el horario, Great Deluge utiliza un umbral de aceptación que varía durante la búsqueda y el enfriamiento simulado permite aceptar algunos empeoramientos.

En un problema de minimización, el enfriamiento simulado acepta siempre una mejora y admite un empeoramiento de magnitud Δ con probabilidad $\exp(-\Delta/T)$. La temperatura T disminuye durante la ejecución. Al principio se aceptan más cambios desfavorables, lo que permite salir de un horario que parece bueno solo dentro de su entorno inmediato. Conforme baja la temperatura, esos cambios se vuelven menos probables y la búsqueda se concentra en reducir el coste. Bellio et al. aplicaron este esquema a CB-CTT y mostraron que la temperatura inicial influye de forma directa en la calidad final [3]. La Fig. 1 recoge ese efecto en una instancia de entrenamiento con 1200 clases. Song et al. ampliaron el método con varios tipos de movimiento y ajustaron la frecuencia con la que se seleccionaba cada uno [4].

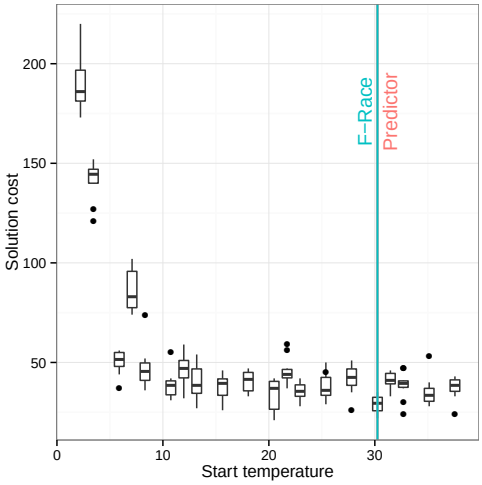


Figura 1. Relación entre la temperatura inicial y el coste final en una instancia de CB-CTT. Cada caja resume varias ejecuciones y un coste menor representa un horario mejor. Las líneas señalan dos formas automáticas de escoger la temperatura. Reproducido de la Fig. 1 de Bellio et al. [3].

El uso de varios tipos de movimiento obliga a decidir cuál conviene aplicar en cada momento. La búsqueda de vecindario variable (Variable Neighborhood Search, VNS) cambia de vecindario cuando el actual deja de producir mejoras. El descenso por vecindarios variables (Variable Neighborhood Descent, VND) sigue una secuencia fijada de búsquedas locales. Las hiperheurísticas llevan esta decisión a un nivel superior y seleccionan el movimiento de bajo nivel que se aplicará a continuación. Otros métodos amplían la búsqueda manteniendo varios horarios a la vez. Abdullah et al., por ejemplo, combinan una población de soluciones con un mecanismo inspirado en electromagnetismo y Great Deluge [5]. La incorporación de componentes

con funciones diferentes conduce a los métodos híbridos, cuyo uso creciente en planificación universitaria recoge la revisión de Abdipoor et al. [6].

V-B. *Métodos híbridos*

Los métodos híbridos combinan técnicas que intervienen en momentos distintos de la búsqueda. Un algoritmo memético, por ejemplo, genera varios horarios y aplica una búsqueda local para mejorar algunos de ellos. En una matheurística, el algoritmo retira temporalmente un conjunto de clases del horario y formula su recolocación como un problema matemático. Kiefer et al. siguen este segundo enfoque mediante una búsqueda adaptativa de gran vecindario (Adaptive Large Neighborhood Search, ALNS) [7]. La parte adaptativa elige qué clases conviene retirar según el rendimiento reciente de cada regla y el modelo exacto vuelve a asignarlas sin alterar el resto del horario. Así se exploran cambios amplios sin perder el control sobre las restricciones.

Otra posibilidad consiste en combinar una población de horarios con una regla de aceptación propia de la búsqueda local. Abdullah et al. asignan a cada horario una fuerza calculada a partir de su penalización y emplean esa información para dirigir Great Deluge [5]. La Fig. 2 muestra el mecanismo sobre horarios concretos. Las soluciones con menor penalización atraen la búsqueda, las peores la alejan y la combinación produce un nuevo horario.

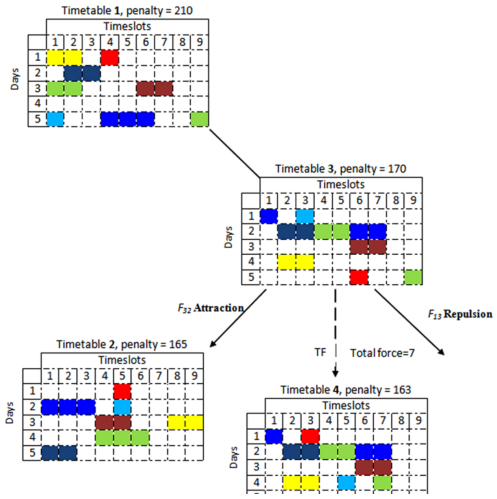


Figura 2. Ejemplo del componente de atracción y repulsión del método híbrido de Abdullah et al. Los horarios con penalizaciones 210 y 165 modifican la búsqueda iniciada desde el horario con penalización 170, que conduce a una nueva solución con penalización 163. Reproducido de la Fig. 2 de Abdullah et al. [5].

V-C. *Nuevas líneas de investigación*

Una línea reciente estudia cómo adaptar el algoritmo a cada instancia antes de comenzar la búsqueda. De Coster et al. describen cada problema mediante 256 características y entrenan selectores que recomiendan el algoritmo más apropiado para cada región del espacio de instancias [8]. Este planteamiento desplaza la pregunta desde qué método es mejor en promedio hacia qué método conviene utilizar ante una estructura concreta de conflictos, aulas y disponibilidad.

El ajuste automático de parámetros también se ha aplicado a LectureKick. Ceschia et al. utilizan `irace` antes de la evaluación final [9]. `irace` no es un modelo de aprendizaje automático. Es un método de configuración automática fuera de línea que compara combinaciones de parámetros sobre instancias de entrenamiento. Las configuraciones menos competitivas se descartan mediante pruebas estadísticas y las restantes reciben nuevas ejecuciones [10]. En este estudio se ajustan, entre otros valores, la temperatura inicial, la velocidad de enfriamiento y las probabilidades de las reglas de selección. Los valores elegidos se mantienen fijos durante los experimentos finales.

A partir de este planteamiento puede estudiarse una adaptación durante la propia búsqueda. Las probabilidades de los movimientos podrían actualizarse según su rendimiento reciente. También puede comprobarse si una configuración obtenida con los horarios de una universidad conserva su utilidad al aplicarla en otra.

VI. El vecindario LectureKick

LectureKick es el principal operador con el que el algoritmo genera un horario vecino. En búsqueda local se denomina operador a la regla que transforma el horario actual mediante cambios como trasladar o intercambiar clases. Ceschia et al. utilizan LectureKick después de una inicialización mediante Hill Climbing y dentro de una fase de enfriamiento simulado [9]. El operador reubica una o dos clases sin incumplir las restricciones obligatorias. Antes de aplicarlo, el programa construye un horario factible y calcula las cuatro penalizaciones blandas del problema.

Cada clase ocupa un hueco definido por un periodo y un aula. En la notación de la Fig. 3, l_i identifica una clase, c_i indica el curso al que pertenece esa clase y s_i representa un hueco del horario. Por ejemplo, $l_2 : c_2$ significa que la segunda clase pertenece al curso c_2. LectureKick parte de l_1 situada en s_1 y propone trasladarla a s_2. Si el destino está libre, basta con mover l_1. Cuando s_2 contiene otra clase l_2, LectureMove solo permite enviarla al hueco liberado s_1. LectureKick añade la posibilidad de colocar l_2 en cualquier hueco libre s_3. De este modo puede alcanzar horarios que no están disponibles mediante un intercambio directo.

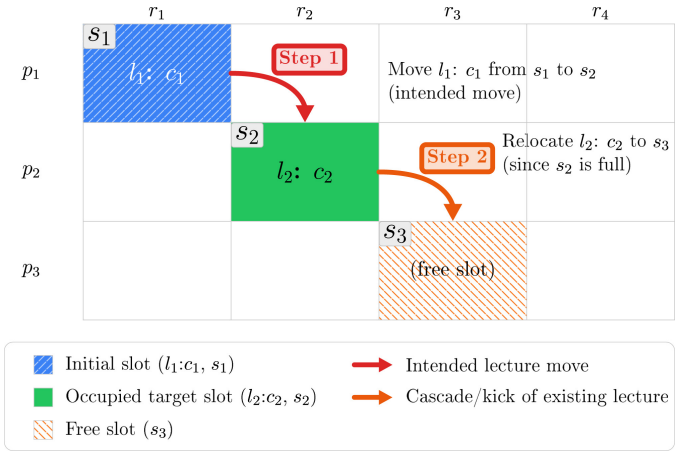
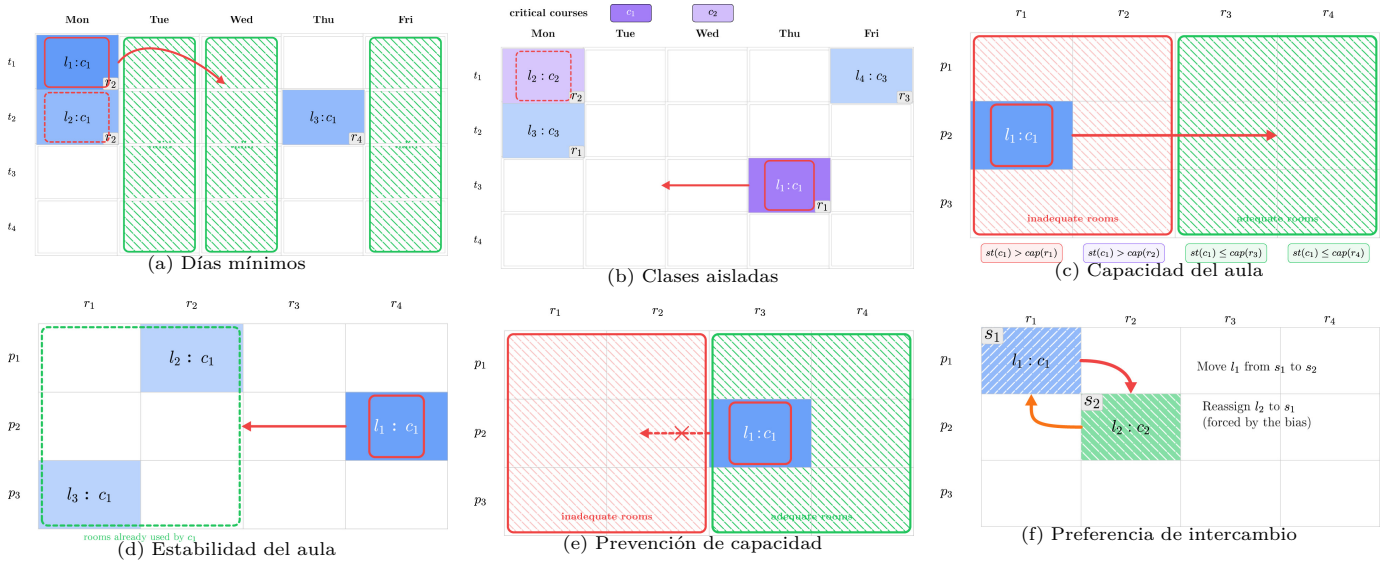


Figura 3. Movimiento LectureKick. El símbolo l_i identifica una clase, c_i su curso y s_i un hueco formado por periodo y aula. La clase l_1 pasa de s_1 a s_2 y, si el destino está ocupado, l_2 puede reubicarse en s_3. Reproducido de la Fig. 1 de Ceschia et al. [9].

Los destinos deben ser distintos, los periodos deben estar disponibles y las clases implicadas no pueden presentar conflictos. Cuando se utiliza s_3, ese hueco debe permanecer libre. Estas condiciones hacen que la regla trabaje únicamente con horarios factibles.

La elección de los movimientos tampoco es uniforme. Un sesgo favorece las clases que todavía no alcanzan el mínimo de días y otro da prioridad a los cursos con riesgo de generar clases aisladas. El algoritmo también prefiere aulas ya utilizadas por el mismo curso y aulas con capacidad suficiente. Un último sesgo aumenta la frecuencia de los intercambios propios de LectureMove. La Fig. 4 reproduce los ejemplos con los que los autores explican estas seis decisiones.



(a) Días mínimos (b) Clases aisladas (c) Capacidad del aula

(d) Estabilidad del aula (e) Prevención de capacidad (f) Preferencia de intercambio

Figura 4. Ejemplos de los seis sesgos de selección empleados con LectureKick. Los cuatro primeros actúan sobre penalizaciones blandas. Los otros dos evitan una penalización de capacidad y favorecen los intercambios. Reproducido de la Fig. 2 de Ceschia et al. [9].

VII. Metodología de reproducción

La reproducción se planteó como una comprobación del método descrito por Ceschia et al. [9]. Se ejecutó el programa publicado sin modificar la búsqueda, de modo que la inicialización con Hill Climbing, el enfriamiento simulado y LectureKick actúan igual que en el artículo. El objetivo no es proponer una variante, sino verificar que el procedimiento puede aplicarse al mismo conjunto de referencia y devolver horarios factibles.

VII-A. Diseño experimental

Se seleccionaron las 19 instancias no redundantes de la formulación UD2 de ITC-2007. Se omitió `comp11` porque su coste objetivo es nulo y `comp15` porque equivale a `comp03` bajo esta formulación. El programa se ejecutó tres veces por instancia con semillas distintas, lo que dio lugar a 57 ejecuciones. En todas ellas se mantuvo la configuración publicada y el presupuesto equivalente al límite virtual de 192 segundos.

Cada ejecución siguió la misma secuencia. El programa construyó primero un horario y utilizó Hill Climbing para alcanzar una solución factible. Después aplicó el enfriamiento simulado con LectureKick hasta agotar el presupuesto de búsqueda. Al finalizar se comprobó que el horario no contenía violaciones obligatorias y se registraron la penalización final y el tiempo. Estas comprobaciones permiten saber si se ha reproducido el funcionamiento esencial del método y si la salida puede analizarse con los mismos criterios que en el artículo.

VII-B. Interpretación y validez

El artículo utiliza 30 repeticiones por instancia, mientras que esta reproducción emplea tres. El número reducido de ejecuciones basta para comprobar el procedimiento y detectar diferencias claras, pero no permite reconstruir con la misma precisión la variabilidad comunicada por los autores.

El sistema operativo y los recursos de cómputo también difieren de los utilizados en el artículo. Esto no impide comprobar la secuencia del algoritmo ni la factibilidad de los horarios, aunque los tiempos solo deben interpretarse de forma orientativa. La Tabla I resume estas diferencias y delimita el alcance de la reproducción [11].

Tabla I
Alcance de la reproducción

Aspecto	Decisión experimental
Método	Programa publicado con Hill Climbing, enfriamiento simulado y LectureKick.
Datos	19 instancias no redundantes de ITC-2007 UD2.
Ejecución	Tres semillas por instancia y 57 ejecuciones en total.
Entorno	Sistema operativo y recursos distintos. Los tiempos son orientativos.
Alcance	Comprobación del método y de la factibilidad, sin reproducir las 30 repeticiones.

VIII. Resultados

IX. Discusión y amenazas a la validez

X. Conclusiones

Referencias

[1] S. Ceschia, L. D. Gaspero, and A. Schaerf, "Educational timetabling: Problems, benchmarks, and state-of-the-art results," *European Journal of Operational Research*, vol. 308, no. 1, pp. 1–18, 2023.

[2] T. Müller, "ITC2007 solver description: A hybrid approach," *Annals of Operations Research*, vol. 172, no. 1, pp. 429–446, 2009.
[3] R. Bellio, S. Ceschia, L. D. Gaspero, A. Schaerf, and T. Urli, "Feature-based tuning of simulated annealing applied to the curriculum-based course timetabling problem," *Computers & Operations Research*, vol. 65, pp. 83–92, 2016.
[4] T. Song, M. Chen, Y. Xu, D. Wang, X. Song, and X. Tang, "Competition-guided multi-neighborhood local search algorithm for the university course timetabling problem," *Applied Soft Computing*, vol. 110, p. 107624, 2021.
[5] S. Abdullah, H. Turabieh, B. McCollum, and P. McMullan, "A hybrid metaheuristic approach to the university course timetabling problem," *Journal of Heuristics*, vol. 18, no. 1, pp. 1–23, 2012.
[6] S. Abdipoor, R. Yaakob, S. L. Goh, and S. Abdullah, "Metaheuristic approaches for the university course timetabling problem," *Intelligent Systems with Applications*, vol. 19, p. 200253, 2023.
[7] A. Kiefer, R. F. Hartl, and A. Schnell, "Adaptive large neighborhood search for the curriculum-based course timetabling problem," *Annals of Operations Research*, vol. 252, no. 2, pp. 255–282, 2017.
[8] A. D. Coster, N. Musliu, A. Schaerf, J. Schoisswohl, and K. Smith-Miles, "Algorithm selection and instance space analysis for curriculum-based course timetabling," *Journal of Scheduling*, vol. 25, no. 1, pp. 35–58, 2022.
[9] S. Ceschia, F. D. Ros, L. D. Gaspero, and A. Schaerf, "Revisiting local search for curriculum-based course timetabling," *Computers & Operations Research*, vol. 195, p. 107621, 2026.
[10] M. López-Ibáñez, J. Dubois-Lacoste, L. P. Cáceres, T. Stützle, and M. Birattari, "The irace package: Iterated racing for automatic algorithm configuration," *Operations Research Perspectives*, vol. 3, pp. 43–58, 2016.
[11] L. M. Weber, W. Saelens, R. Cannoodt, C. Soneson, A. Hapfelmeier, P. P. Gardner, A.-L. Boulesteix, Y. Saeys, and M. D. Robinson, "Essential guidelines for computational method benchmarking," *Genome Biology*, vol. 20, p. 125, 2019.